FIRM
Build Specification for Claude Code

Bonded-quote enforcement and a public fill-quality ledger for Uniswap v4 swaps on Monad

Hackathon	Monad Metropolis (online, 1 Sep to 13 Oct 2026). Submission deadline 13 Oct 2026, 11:59 PM ET (reported rules section 4.2; confirm on the platform)
Track	Track 1: Onchain Finance and Trading (one submission, one track)
Target network	Monad Testnet, chain ID 10143 (testnet only; mainnet read-only at most)
Document version	1.1, revised 6 October 2026 after a verification pass (supersedes 1.0)
Audience	Claude Code (primary builder) and the human team (reviewers)
Status legend	[V] verified against a primary source | [S] second-hand source | [A] design assumption: test in a spike | [NEW] added or changed in v1.1

How to use this PDF
Read Section 0 first, then the Revision notes. Section 16 now contains a 7-day plan, because only about seven days remain before the deadline. Appendix A is a ready-made CLAUDE.md for the repository root.

Revision notes (v1.1)

This version corrects and extends v1.0 after checking its claims against primary sources on 6 October 2026. Nothing below changes the core idea (same-transaction quote versus fill). Changes are marked **[NEW]** in the body.

What changed

#	Change	Where
1	**Timeline.** About 7 days remain, not six weeks. Added a 7-day plan with a scope cut; P1 items other than the minimal canary are deferred.	16.1
2	**Missing rule added:** the README must disclose that AI coding tools were used (reported rules section 4.1.4, treated as a disqualification risk). Also one submission, one track.	0.1, 17
3	**Tracks and prizes:** the four track names, the $25K grand champion and the per-track split ($10K to each of the top 3, reported) are now stated. One event page says $30K overall; treat $25K as correct and confirm.	2.1
4	**Bounties:** Envio, Mera, Tencent and Alchemy are not confirmed on the official lists I could read. Confirmed sponsors added: Agora, Nansen, Chainlink, Privy, Alibaba Cloud, Kimi, Dynamic, Perpl, Kuru, MetaMask. Privy (passkeys, testnet) added as an alternative to Mera.	2.2
5	**Envio endpoint corrected:** use monad-testnet.hypersync.xyz (and the matching HyperRPC URL) from Envio's page; the 10143 alias in v1.0 was not confirmed. Config chain id format must be checked against what envio init generates.	5.4, 9
6	**Evidence updated:** 0x figures verified. Added the 26.4% 'likely malicious' and 19.4% 'safe' shares so the 54.2% is not quoted alone. The arXiv paper is real but is about propAMM spoofing, not v4 hooks.	3.2
7	**Quoter risk made explicit:** Uniswap documents the Quoter as not intended for onchain calls (revert-based, expensive). S1 now has gas pass criteria and a per-leg gas budget.	4.2, 8.4, 15
8	**New mitigation:** optional advertisedOut per route and LegSkipped reason 4, so a hook that quotes differently off-chain (eth_call) than in-transaction is skipped, not just not struck.	8.4
9	**Hook flag bits:** canonical layout recorded (BEFORE_SWAP 1<<7, AFTER_SWAP 1<<6, returns-delta flags 1<<3 and 1<<2), cross-checked against Counter's 0x0ac0. A popular tutorial shows 1<<0; do not use it.	5.5, 9.3
10	**Dates and arithmetic:** 'submit with a day of margin' now means by 12 Oct.	17.1

Verification log (6 October 2026)

Item	Result	Status
Testnet chain ID, client version, reset date, explorers, faucet	Match Monad docs	[V]
Three public RPCs and their rate limits	Match Monad docs exactly (50 / 25 for eth_call / 300 per 10 s / 20 rps)	[V]
Canonical contracts (WMON, Permit2, Multicall3, CreateX, deterministic deployer, x402 proxies)	Match Monad docs	[V]
Uniswap v4 testnet addresses (PoolManager, Quoter, StateView, PositionManager, Universal Router, etc.)	All 9 match the Monad hooks guide table; mainnet PoolManager also matches	[V]
Envio testnet support	First-class (HyperIndex, HyperSync, HyperRPC); endpoint hostnames differ from v1.0	[V]
0x post: 84,163 hooks, 54.2% malicious, hook stats	Match the post dated 14 Sep 2026 (data as of 11 Sep)	[V]
arXiv 2609.38056	Exists; 12.5% to 45.7% Monad propAMM share confirmed; suggests onchain router	[V]
Metropolis tracks, prizes, dates	Match official page; rules details are second-hand	[V] / [S]
Testnet acceptable; open source mandatory	Quoted from rules v3.0 by several participant repos	[S]
V4Quoter inside a transaction (S1)	Not verifiable by reading; needs a fork test. Uniswap says not for onchain use	[A]

Item	Result	Status
Initialize event signature, exact flag bits (S4)	Layout cross-checked via Counter 0x0ac0; still read Hooks.sol	**[A]**
Kuru, Perpl, Agora AUSD, Chainlink CRE on testnet	Not verified; remain out of scope	**[A]**

Contents

Revision notes (v1.1)	2
Contents	4
0. Instructions for Claude Code	5
1. Project overview	6
2. Hackathon context	7
3. Problem and evidence	9
4. Solution and design principles	10
5. Verified testnet environment	12
6. System architecture	14
7. Repository layout	15
8. Smart contracts specification	16
9. Indexer (Envio HyperIndex)	22
10. quote-api service	24
11. Agents	25
12. Frontend (apps/web)	27
13. Testing and benchmark plan	29
14. Configuration and operations	30
15. Spikes and fallbacks	31
16. Milestones, acceptance criteria and prompts	32
17. Submission checklist and demo script	35
18. Risks and mitigations	36
19. References	37
Appendix A: CLAUDE.md (copy to the repository root)	38
Appendix B: .env.example	38
Appendix C: Glossary	38
Appendix D: Questions for the human, with defaults	39

0. Instructions for Claude Code

You are the lead engineer for FIRM. This document is your complete specification: what we are building, why, the exact components, the interfaces between them, the order of work, and how to prove it works. Read all of it before writing code, then create the repository and copy Appendix A into CLAUDE.md.

0.1 Operating rules

1. Follow the milestone order in Section 16. Milestone M0 contains the spikes that test the riskiest assumptions. Do them first. If one fails, use the fallback in Section 15 and record the result in docs/SPIKES.md.
2. Never invent addresses, ABIs, event signatures or API shapes. Use the addresses in Section 5. For everything marked [A], read the upstream source (Uniswap v4-core and v4-periphery, the Monad hooks guide) and confirm by test.
3. Testnet only. Everything deploys to Monad Testnet (chain ID 10143). Mainnet access, if any, is read-only. Never put a private key in git; use the Foundry keystore or .env (Appendix B).
4. Deterministic code decides money and status. An LLM may write reports and drive a demo agent, but it must never decide whether a swap executes or what status a hook gets.
5. Test fixtures that misbehave are quarantined. Put them in contracts/test-fixtures/, name them clearly (for example FixtureContextHook), deploy them to testnet only, never register them in any public hook list, and say so in the README.
6. Write tests with every contract and service. Run forge test and the TypeScript tests before each commit. Commit small and often with clear messages (the commit history must span the build window).
7. Keep the docs alive: README.md, docs/SPIKES.md, DEPLOYMENTS.md, THREAT_MODEL.md and RESULTS.md, as you go.
8. Ask the human only when blocked on credentials, funds, accounts or a product decision. Otherwise choose the default stated in this document and move on.
9. [NEW] **AI-tool disclosure.** The README must state that AI coding tools (Claude Code) were used and name them. The reported rules (section 4.1.4) make this mandatory; omission risks disqualification.
10. [NEW] **Scope discipline.** About seven days remain. Follow the 7-day plan in 16.1 and cut from the end of the P1 list before touching P0. Never cut M7 (benchmark) or M8 (submission).

0.2 Decisions already made (do not reopen)

Topic	Decision
Network	Monad Testnet (10143). Contracts use mock stable tokens, not Agora AUSD.
Core mechanism	Same-transaction quote-versus-fill check. No oracle and no CEX price.
Language and tooling	Solidity 0.8.26, Cancun EVM, Foundry v1.8.0 (pinned by the Monad guide). TypeScript on Node 22, pnpm workspaces, viem.
Indexer	Envio HyperIndex on Monad Testnet, with the viem poller (F4) as a pre-agreed fallback.
Frontend	Next.js (App Router), React, Tailwind, wagmi v2 + viem, TanStack Query.
Agents	TypeScript. LLM access through any OpenAI-compatible endpoint configured by env vars.
Licence	MIT (open source is mandatory under the reported rules, section 7.2).
Admin model	Ownable2Step owner for the hackathon, documented as a centralisation trade-off in THREAT_MODEL.md.
Track	[NEW] Track 1 only. One project, one track (reported rules section 2.5).

1. Project overview

1.1 What FIRM is, in plain words

When you swap tokens, an app shows a quote such as 'you will receive about 999 USDC'. On Uniswap v4, liquidity pools can attach a hook: a custom contract that runs during the swap and can change the amounts. A badly behaved hook can look good in the quote and then take a hidden fee when the swap really executes. Nothing in today's routers remembers which hooks did that, so the next user gets hit too.

FIRM turns the quote into a promise. For every swap it asks the pool for a quote and then executes at the identical state inside the same transaction. If the settled amount is below the promise, FIRM reverts only that leg (the user's funds are untouched), writes a public strike against the hook, and falls back to the next pool. A separate agent probes new hooks with dust swaps before real users arrive, and everything lands in an onchain scoreboard that any other router can read.

1.2 Worked example (this becomes the live demo)

Pool	Quoted output for 1,000 tUSD	What really happens at execution
Pool B (fixture hook that behaves differently on the real path)	999.0 (best quote)	Hidden fee of about 18%: only about 819.2 delivered
Pool C (honest hook)	998.2	Delivers 998.2
Pool A (no hook, baseline)	997.5	Delivers 997.5

- Naive router: picks Pool B (highest quote) and the user receives about 819.2, losing roughly 180.
- FIRM: quotes Pool B at 999.0, executes, sees 819.2 is below the floor, reverts that leg, records a strike, moves to Pool C, and the user receives 998.2.
- The scoreboard shows Pool B's hook turning red, and the event appears on the dashboard within seconds.

1.3 The five components

#	Component	One-line purpose
1	Smart contracts (FirmRouter, Scoreboard, CanaryRunner, RawRouter, optional BondVault)	Enforce the promise and keep the public record
2	Indexer (Envio)	Turn events into a queryable history and live leaderboard
3	quote-api (TypeScript service)	Find candidate pools and return ordered routes with floors
4	Agents: canary-agent, firm-mcp + trader-agent, receipt-writer	Detect suspect hooks early; show an AI trader that cannot be cheated
5	Web app (Next.js)	Swap with protection, leaderboard, hook detail, arena, integration guide

1.4 What success looks like

- A judge opens the web app, sees the same trade run through a naive router and through FIRM, and understands the value in about ten seconds.
- All contracts are deployed and verified on Monad Testnet, with addresses and transaction hashes in DEPLOYMENTS.md.
- RESULTS.md contains measured numbers: catch rate, false positives, user loss with and without FIRM, gas overhead.
- The repository is public (MIT), documented, reproducible with one redeploy script, and honest about its limits.

2. Hackathon context

2.1 Metropolis facts

Fact	Detail	Status
Format	Six-week online hackathon run by the Monad Foundation, 1 Sep to 13 Oct 2026; judging 14 to 27 Oct; winners 3 Nov	[V]
Deadline	[NEW] 13 Oct 2026, 11:59 PM ET (the project page countdown matches; rules section 4.2 reported). Rolling submission; the version at the deadline is judged	[S]
Tracks	[NEW] (1) Onchain Finance and Trading; (2) Consumer Products and Payments; (3) Social, Attention and Culture; (4) Trust, Identity and AI Infrastructure	[V]
Prizes	$250K+ headline pool: $30K per track, a $25K grand champion, plus sponsor bounties. Rules v3.0 reportedly pay $10K to each of the top 3 per track and cite a $145K cash total. One event page says '$30,000 overall winner'; treat $25K as correct and confirm	[V] / [S]
Our track	Track 1. The page says it fits teams who have shipped a trading, lending or market-making product before (examples: onchain order books, per-block perps funding, undercollateralised lending)	[V]
Judges	Mostly investors and operators (Paradigm, Electric Capital, Dragonfly, Pantera, Galaxy, Castle Island, Primitive and the Monad founders)	[V]
What to submit	Working product, public project profile, demo, short write-up, code link	[V]
Teams	[NEW] Solo or 1 to 5 members; exactly one project and one track per team (disqualifying otherwise)	[S]
Mainnet or testnet	Either accepted; contract addresses or tx hashes required; explain why Monad's capabilities are used (rules section 9.2, quoted by several participant repos)	[S]
Open source	Mandatory under rules section 7.2 (OSI licence, public GitHub during and after the event); the marketing FAQ says 'encouraged'. The rules control	[S]
AI disclosure	[NEW] README must state which AI coding tools were used (rules section 4.1.4, reported)	[S]
Originality	Work shown must be built inside the window; disclose pre-existing code (rules section 4.1 clause 4, reported)	[S]
Review	Submissions reviewed non-confidentially; judges may build similar products	[S]

2.2 Bounties this project can realistically target on testnet

Confirmed sponsor names (official listings): Agora, Nansen, Chainlink, Privy, Alibaba Cloud, Kimi, Dynamic, Perpl, Kuru, MetaMask and others. Amounts and exact rules must be read on hackathon.monad.xyz. Do not count on a bounty until its rules are read.

Bounty	Why it fits	Confidence
Envio ($1K, 'Best Use of Envio' in v1.0)	HyperIndex powers the scoreboard and leaderboard; Monad Testnet is first-class on Envio. Bounty listing not confirmed in this pass	Medium [A]
Kimi, Alibaba Cloud (Qwen) LLM credits	OpenAI-compatible APIs for the canary report and trader agent. Tencent was listed in v1.0 but is not confirmed	Medium [A]
[NEW] Privy	Passkey login for the swap UI. A participant repo reports Privy supports passkeys on testnet and subsidises Monad testnet usage (second-hand)	Medium [A]
Monad Foundation Mera ($2.5K each)	Passkey login (P2). Not confirmed in this pass; Mera is client-side, confirm testnet behaviour	Low-Medium [A]
[NEW] Nansen	Possible fit if the project uses Nansen data; read the bounty rules first. Not planned	Low [A]
Alchemy credits	Fallback RPC. Some participants report Alchemy and Chainstack faucets need real ETH; QuickNode's Monad faucet is free	Low [A]
MetaMask Agent Wallet plugin ($2.5K)	Optional P2; requires Early Access Program access	Low [A]

Out of scope bounties
Kuru, Chainlink CRE, Perpl and Agora AUSD remain out of scope because testnet availability was not verified. Add them only if the human confirms they work on testnet.

3. Problem and evidence

3.1 The problem

Aggregators and routers rank liquidity sources by quoted output. A Uniswap v4 hook can change swap amounts and can behave differently when simulated for a quote than when really executed. Quote spoofing therefore pays: the best quote wins the order and the user absorbs the hidden fee. Defences today are manual blocklists, slippage settings and onchain routing, but none keeps a shared, public, machine-readable record of which sources honour their quotes.

3.2 Evidence (cite exactly as attributed)

Claim	Source	Notes	Status
0x analysed 84,163 Uniswap v4 hooks across 6 chains: 19.4% safe, 54.2% malicious, 26.4% likely malicious (data as of 11 Sep 2026)	0x post 'Uniswap v4 hooks were a mistake', 14 Sep 2026	Quote the 54.2% together with the 26.4% 'likely malicious' share. Uniswap's founder disputes the framing, noting the count includes hooks with little activity (via Crypto Briefing). Present as 0x's classification	[V]
One flagged hook (Base, ETH/NVDAc) charged a hidden fee on 3,946 of 6,516 fills (60.6%), median 18% when charged, about $143,037 total	Same 0x post	The post names the hook address; FIRM's docs should not. 0x also reports trades delivering up to 50% less than quoted	[V]
0x describes hooks that price swaps one way in quote requests and another at settlement, some by inspecting the EVM environment	Same 0x post	Motivates both the same-transaction check and the static-analysis flags in the canary	[V]
0x says routers need to ensure a pool's quoted amount matches its actual execution	Same 0x post	Exactly FIRM's design goal	[V]
Monad's co-founder publicly called aggregator quote spoofing a huge hidden tax	The Monad Pulse #047; Kucoin news	Ecosystem relevance. Not re-verified in this pass	[S]
Uniswap v4 is deployed on Monad mainnet (chain 143); PoolManager 0x188d586ddcf52439676ca21a244753fa19f9ea8e	Uniswap deployments page; Monad hooks guide	Matches the Monad guide	[V]
Universal Router executes the supplied route and does not certify hook safety	Monad hooks guide	Exactly the gap FIRM fills	[V]
In one Monad pair (MON/USDC), propAMMs' share of volume against AMMs rose from 12.5% to 45.7%	arXiv 2609.38056 (Solmaz, Heimbach, Milionis)	The paper studies propAMMs, not v4 hooks; use it for 'quote honesty matters on Monad'	[V]
The paper suggests an onchain router that queries each venue inside the swap transaction would remove the quote-to-fill gap	arXiv 2609.38056, concluding discussion	Academic backing for the same-transaction design	[V]

3.3 Why existing fixes fall short

- Slippage tolerance is a single global setting and users set it loosely.
- Manual curation (blocklists inside aggregators) is private to each company and slow.
- Onchain routing shortens the delay between quote and fill but does not punish a source that cheats at execution time.
- Markout dashboards measure price quality against a reference price, not broken promises, and cover propAMMs rather than v4 hooks.
- Nobody publishes a shared standard for measuring fill quality onchain, which 0x itself says the industry lacks.

4. Solution and design principles

4.1 The core insight

Same-transaction quote versus fill
Inside one transaction the pool's state cannot change except through the pool and its hook. So if FIRM asks the pool for a quote and then executes the swap in the same transaction, any difference between quoted and settled output is caused by the hook's behaviour, not by the market. This needs no oracle, no CEX reference price and no dispute process. **[A]** This is our design inference and must be validated in spikes S1 and S3.

4.2 Mechanism in six steps

1. The caller passes an ordered list of candidate routes (best quote first, from quote-api) to FirmRouter.
2. For each route FirmRouter checks the hook's status on the Scoreboard and skips BLOCKED hooks (and caps UNRATED hooks).
3. It calls the V4 Quoter inside the transaction to get quotedOut, and computes floor = quotedOut minus the tolerance for that status.
4. It executes the swap inside a try/catch with a gas cap. The execution function itself reverts if the output is below the floor, carrying the settled amount in the revert data.
5. On shortfall (or any execution failure after a successful quote) the router, in the outer call frame, records a strike on the Scoreboard and moves to the next route. The user's funds are untouched.
6. On success it pays the user, records the fill, and finishes. If every route fails it refunds the input and returns zero output so the strikes persist.

[NEW] What the in-transaction Quoter does and does not catch
The V4 Quoter calls the PoolManager as the swap caller, so a hook sees the Quoter's address as the 'sender' on the quote path and FirmRouter's address on the real path. A hook that spoofs for quote requests (the pattern 0x describes) is therefore caught by the in-transaction comparison. Do **not** replace the Quoter with a self-simulation through FirmRouter: the hook would then see the same sender on both paths and Quoter-specific spoofing would go unseen. What the check cannot catch: a hook that is honest to both the Quoter and FirmRouter but cheats other callers (the sender-aware case, covered by CanaryRunner), and a hook that is simply expensive but consistent. Uniswap documents the Quoter as revert-based and not intended for onchain calls, so spike S1 must measure its gas on Monad, where gas is charged on the limit.

4.3 Supporting mechanisms

- **Scoreboard:** per-hook statistics and a status (UNRATED, NORMAL, WATCH, BLOCKED) readable by any contract.
- **CanaryRunner:** a permissionless onchain probe that performs the same quote-versus-fill check from rotating sender contracts, so evidence is computed onchain and the keeper agent is only a trigger.
- **RawRouter:** a deliberately naive comparator router (no checks) used for the side-by-side demo and the benchmark.
- **BondVault (P2):** sources that post a bond are repaid any within-tolerance shortfall from the bond; large breaches slash it.

4.4 Non-goals

- Protecting against sandwich attacks or other MEV, or against manipulation of honest pools.
- Protecting users who bypass FIRM.
- Native-token (address zero) currencies, fee-on-transfer tokens, multi-hop routes and split orders in the MVP.
- A trust-minimised attester set. For the hackathon the owner and a keeper address are trusted roles, documented as such.

4.5 Known limits (state them in the README, never hide them)

- A hook can behave honestly toward FirmRouter and cheat other callers. CanaryRunner probes from several sender identities to catch this, but a determined hook could try to recognise probes (the arXiv paper documents propAMMs penalising callers by address and bytecode fingerprint). This is an arms race, not a solved problem.
- A hook that behaves identically on quote and real paths but is simply expensive is not a spoof; FIRM only measures promise-keeping, not price quality.
- Honest hooks with legitimate caller-specific discounts may look suspicious and land in WATCH.
- On testnet we use our own fixture hooks, so the evidence is controlled, not field data.
- Monad charges gas on the gas limit rather than gas used, so fallback chains cost more than their actual execution suggests.
- **[NEW]** Fixtures are written by us, so catching them proves the mechanism works, not that real malicious hooks are caught. Say so in RESULTS.md.

5. Verified testnet environment

Sections 5.1 to 5.4 were re-checked on 6 October 2026 against Monad's documentation, the Monad hooks guide and Envio's chain page. Re-check shared-infrastructure.json in the example repo before relying on an address.

5.1 Network

Item	Value	Status
Chain ID / currency	10143 / MON (18 decimals)	[V]
Client version	v0.15.2, revision MONAD_NINE. Testnet was reset from genesis on 2025-12-16, so it can be reset again	[V]
RPC 1 (QuickNode)	https://testnet-rpc.monad.xyz (wss available). 50 rps; 25 rps for eth_call and eth_estimateGas; batch 100; archive yes	[V]
RPC 2 (Ankr)	https://rpc.ankr.com/monad_testnet. 300 req/10 s and 12,000 req/10 min; batch 100; no archive; debug_* blocked	[V]
RPC 3 (Monad Foundation)	https://rpc-testnet.monadinfra.com (wss available). 20 rps; batch not allowed; archive yes	[V]
Explorers	https://testnet.monadvision.com and https://testnet.monadscan.com (Envio's page also lists testnet.monadexplorer.com)	[V]
Faucet	https://faucet.monad.xyz. Participants also report QuickNode's Monad faucet works for free tokens	[V] / [S]
Gas model	Gas is charged on the gas limit, not gas used. Set explicit gas limits on scripts and cap per-leg gas in the router	[V]
P256 precompile	EIP-7951 P256VERIFY enabled (relevant to passkeys); address 0x0100 per a third-party doc	[V] / [S]

5.2 Canonical contracts on testnet

Name	Address
Wrapped MON	0xFb8bf4c1CC7a94c73D209a149eA2AbEa852BC541
Permit2	0x000000000022D473030F116dDEE9F6B43aC78BA3
Multicall3	0xcA11bde05977b3631167028862bE2a173976CA11
CreateX	0xba5Ed099633D3B313e4D5F7bdc1305d3c28ba5Ed
Foundry Deterministic Deployer (CREATE2)	0x4e59b44847b379578588920ca78fbf26c0b4956c
x402 ExactPermit2Proxy (optional, agents)	0x402085c248EeA27D92E8b30b2C58ed07f9E20001
x402 UptoPermit2Proxy (optional, agents)	0x4020A4f3b7b90ccA423B9fabCc0CE57C6C240002

All entries verified against docs.monad.xyz/developer-essentials/testnets on 6 Oct 2026 [V]. That page also lists SafeSingletonFactory, ERC-6492 validator, ERC-4337 EntryPoints v0.6 to v0.9 and Safe v1.4.1 contracts, not needed here.

5.3 Uniswap v4 on testnet (Monad-maintained)

Not official deployments
These are independent testnet deployments maintained by Monad. They differ from mainnet addresses. Source: monad-developers/uniswap-v4-hooks-example, shared-infrastructure.json (docs pin commit c3cc78a88f10e5ba141cd6fb632f31db83c0daf8). The PoolManager has no owner or protocol-fee controller.

Contract	Address (chain 10143)
PoolManager	0x451D64ab3b650040d2aE1886602b97ed6eDc643d
Universal Router 2.1.1	0x1b7bFCd2870329B987191910D85c22C7287f3c22
V4 Quoter	0x869834d127b230283fe63E0d0A9bEB67216a94C7

Contract	Address (chain 10143)
StateView	0xB639209539c61BaF67AC04876315786F8D0b153c
PositionManager	0x3Bb14E3D0Cd50aBe3EdACa06d06c29C78676C31A
PositionDescriptor	0x1548b69E74BdD834c2e552a5c673acf921Aa7B11
UnsupportedProtocol	0xF5fa008BdB979427545c40fecE80d8cdEE4177C2

Mainnet (read-only reference): chain 143, RPC https://rpc.monad.xyz, Uniswap v4 PoolManager 0x188d586ddcf52439676ca21a244753fa19f9ea8e [V]. The hooks guide also warns that a hook callback's 'sender' is the caller of the manager (often a router), not the end user. Do not treat it as user identity.

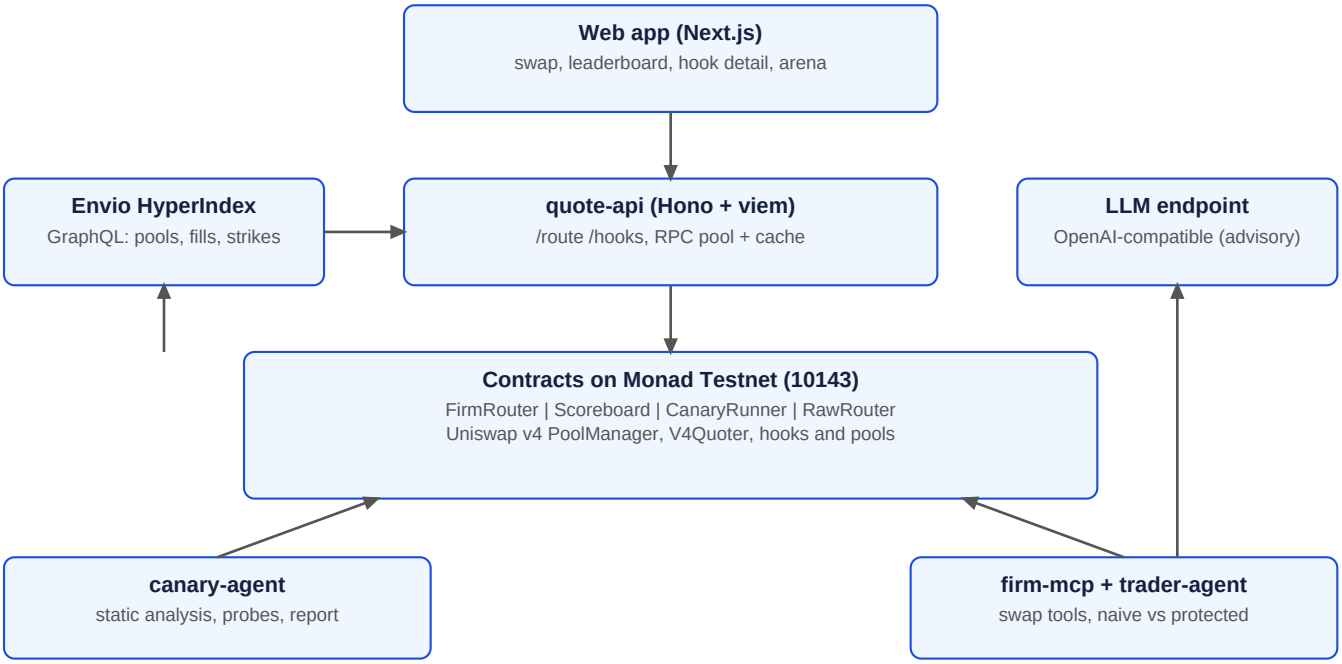
5.4 Tooling pins and indexing

Item	Value	Status
Foundry	v1.8.0 (pinned by the Monad hooks guide)	[V]
Solidity / EVM	0.8.26 / Cancun (from the example repo's foundry.toml)	[V]
Hook example repo	github.com/monad-developers/uniswap-v4-hooks-example (docs pin commit c3cc78a88f10e5ba141cd6fb632f31db83c0daf8). Scripts: DeployHook, DeployMockTokens, PoolLifecycle, ReadState. Needs Node/npm, Python 3 and jq	[V]
Hook address mining	The example uses NamespacedHookMiner.find with the CREATE2 factory to find a salt whose address has the needed permission bits. Counter's bits are 0x0ac0	[V]
[NEW] Envio testnet HyperSync	https://monad-testnet.hypersync.xyz (v1.0 listed 10143.hypersync.xyz as an alias; not confirmed, do not use)	[V]
[NEW] Envio testnet HyperRPC	https://monad-testnet.rpc.hypersync.xyz (read-only, for data-heavy queries)	[V]
[NEW] Envio config	Envio's example uses 'id: monad-testnet'; v1.0 said chain id 10143. Use whatever envio init generates. Envio documents HyperSync API tokens: check whether one is required for hosted use	[V] / [A]
Envio mainnet HyperSync	https://143.hypersync.xyz (not needed for testnet-only work)	[S]

5.5 Not verified (do not rely on these)

- Kuru, Perpl, Agora AUSD and Chainlink CRE availability on testnet.
- Mera behaviour on testnet; MetaMask Agent Wallet plugin access (Early Access Program).
- Whether the V4 Quoter can be called from a contract inside a transaction at acceptable gas on Monad (spike S1).
- Exact Initialize event signature (expected: Initialize(PoolId indexed id, Currency indexed currency0, Currency indexed currency1, uint24 fee, int24 tickSpacing, IHooks hooks, uint160 sqrtPriceX96, int24 tick); read IPoolManager.sol to confirm).
- [NEW] Permission-bit layout. Expected from v4-core: BEFORE_SWAP = 1<<7, AFTER_SWAP = 1<<6, BEFORE_SWAP_RETURNS_DELTA = 1<<3, AFTER_SWAP_RETURNS_DELTA = 1<<2, BEFORE_ADD_LIQUIDITY = 1<<11, BEFORE_REMOVE_LIQUIDITY = 1<<9. This is consistent with Counter's 0x0ac0 (0x800 + 0x200 + 0x80 + 0x40). Confirm in Hooks.sol. Some tutorials show BEFORE_SWAP = 1<<0; that is wrong for the deployed core.

6. System architecture



Solid arrows show calls or data flow; contracts make all fund and status decisions.

6.1 Data flow for one protected swap

1. User picks a pair and amount in the web app. The app asks quote-api for candidate routes.
2. quote-api finds pools for the pair (from the indexer), drops hooks the Scoreboard marks BLOCKED, batches V4 Quoter calls, and returns routes ordered best quote first.
3. The app simulates FirmRouter.swapExactIn with eth_call to preview the outcome, then the user signs.
4. FirmRouter runs the six-step mechanism from Section 4.2 and emits Filled / Rejected / LegSkipped events.
5. Envio indexes the events. The leaderboard and hook pages update within seconds. The Swap page parses the receipt logs to draw the timeline.

6.2 Trust boundaries

Component	Trusted for	Not trusted for
FirmRouter, Scoreboard, CanaryRunner	All fund-affecting and status-affecting decisions	Nothing offchain participates in these decisions
Owner key	Reporter list, parameters, reinstating a BLOCKED hook	Cannot move user funds
canary-agent	Triggering probes and writing reports	Cannot set status; evidence is computed onchain by CanaryRunner
LLM	Prose reports and a demo trader	Never decides execution, status or parameters
quote-api	Ordering candidate routes	Router re-quotes and re-checks everything onchain
Indexer	Display only	Not used by contracts

7. Repository layout

```
firm/
  CLAUDE.md                      # copy of Appendix A
  README.md THREAT_MODEL.md DEPLOYMENTS.md RESULTS.md LICENSE (MIT)
  docs/SPIKES.md                 # results of spikes S1-S4 and any fallbacks used
  deployments/10143.json         # single source of truth for addresses (written by scripts)
  contracts/                     # Foundry project (solc 0.8.26, cancun)
    foundry.toml remappings.txt lib/ (v4-core, v4-periphery, openzeppelin, forge-std)
    src/
      FirmRouter.sol Scoreboard.sol CanaryRunner.sol ProbeSender.sol
      RawRouter.sol BondVault.sol (P2)
      base/SwapExecutor.sol      # shared unlock/swap/settle/take logic
      libraries/RouteLib.sol     # Route struct helpers, bps math
      interfaces/ (IScoreboard.sol, IFirmRouter.sol, IV4QuoterLite.sol)
      test-fixtures/             # QUARANTINED test hooks and mock tokens
        FixtureHonestHook.sol FixtureContextHook.sol FixtureRandomFeeHook.sol
        FixtureSenderAwareHook.sol FixtureGasGriefHook.sol FixtureRevertHook.sol
        MockToken.sol
    script/ Deploy.s.sol DeployFixtures.s.sol SeedPools.s.sol Redeploy.sh
    test/                        # unit, fuzz, invariant, fork tests
  indexer/                       # Envio HyperIndex: config.yaml, schema.graphql, src/handlers
  services/
    quote-api/  canary-agent/  firm-mcp/  trader-agent/
    shared/                      # abis, deployments loader, rpc pool, logger
  apps/web/                      # Next.js app
  bench/                         # benchmark runner that writes RESULTS.md
  .env.example
```

Use pnpm workspaces for services, apps and bench. Generate ABIs from Foundry artifacts into services/shared/abis with a script so TypeScript always matches the deployed contracts.

8. Smart contracts specification

8.1 Conventions

- Solidity 0.8.26, custom errors, NatSpec on every external function, events for every state change.
- Use OpenZeppelin for Ownable2Step, ReentrancyGuard and SafeERC20. Use Uniswap v4-core types (PoolKey, PoolId, BalanceDelta, Currency, IHooks) and IPoolManager / IUnlockCallback.
- Money amounts are uint128 at the API boundary and uint256 internally. All bps math uses 10_000 as denominator.
- ERC-20 only. Reject native currency (address zero) pools. Route tokens must match across legs.
- Hookless pools (hooks == address(0)) are always allowed, always treated as NORMAL, and never written to the Scoreboard.

8.2 Shared base: SwapExecutor

An abstract contract used by FirmRouter, RawRouter and ProbeSender so the swap code path is identical everywhere. **[A]** The settle and take sequence below must be confirmed against v4-periphery examples.

```
abstract contract SwapExecutor is IUnlockCallback {
    IPoolManager public immutable poolManager;
    struct Leg { PoolKey key; bool zeroForOne; bytes hookData; uint128 amountIn; }
    // Runs the swap via poolManager.unlock(). Returns exact output received.
    function _swapExactIn(Leg memory leg) internal returns (uint256 amountOut);
    // IUnlockCallback: only callable by poolManager.
    // 1. delta = poolManager.swap(key, SwapParams{zeroForOne, -int256(amountIn),
    //            price limit = MIN/MAX +/- 1}, hookData)
    // 2. input side:  poolManager.sync(cIn); cIn.transfer(poolManager, amountIn);
    //                 poolManager.settle();
    // 3. output side: poolManager.take(cOut, address(this), outAmount)
    // 4. return abi.encode(outAmount)
    function unlockCallback(bytes calldata data) external returns (bytes memory);
}
```

8.3 Scoreboard

Public ledger of hook honesty. Writers are an allow-list of reporters (FirmRouter, CanaryRunner). Anyone can read.

```
enum Status { UNRATED, NORMAL, WATCH, BLOCKED }
struct HookStats {
    uint32 fills; uint32 strikes; uint32 probes;
    uint32 probeStrikes; uint32 cleanProbes; uint32 cleanStreak;
    uint32 recentStrikes; uint16 cleanProbeSenders;
    uint64 windowStart; uint64 lastStrikeAt; uint64 firstSeenAt;
    uint128 cumShortfallBps;
    Status status;
}
interface IScoreboard {
    function statusOf(address hook) external view returns (Status);
    function toleranceBps(address hook) external view returns (uint16);
    function unratedCap() external view returns (uint128);
    function stats(address hook) external view returns (HookStats memory);
    // reporter-only
    function recordFill(address hook, bytes32 poolId, uint256 quotedOut, uint256 settledOut) external;
    function recordStrike(address hook, bytes32 poolId, uint256 quotedOut,
                          uint256 settledOut, bool execFailed) external;
    function recordProbe(address hook, bytes32 poolId, address probeSender,
                         uint256 amountIn, uint256 quotedOut, uint256 settledOut) external;
    // owner-only
    function setReporter(address who, bool allowed) external;
    function setParams(Params calldata p) external;   // bounded ranges
    function reinstate(address hook) external;        // BLOCKED -> WATCH
}
```

Default parameters (owner-tunable within bounds)

Parameter	Default	Meaning
TOL_NORMAL_BPS / TOL_UNRATED_BPS	10 / 10	Allowed shortfall versus the in-transaction quote. Honest pools should show about 0, so this only absorbs rounding
TOL_WATCH_BPS	2	Tight tolerance for hooks that already have a strike
SEVERE_BPS	200	A shortfall at or above this (or any execution failure after a good quote) blocks the hook at once
STRIKE_WINDOW	3600 s	Window for counting repeated strikes
BLOCK_AFTER_STRIKES	3	Strikes inside the window that move a hook to BLOCKED
RATING_MIN_PROBES / RATING_MIN_SENDERS	6 / 3	Clean probes and distinct probe senders needed to move UNRATED to NORMAL
WATCH_RECOVERY	200 clean fills and 24 h	Needed to move WATCH back to NORMAL
UNRATED_CAP	100e18 (100 tokens)	Maximum input size routed through an UNRATED hook

Status transition rules (implement exactly; test every edge)

1. Shortfall bps = max(0, quoted - settled) * 10_000 / quoted. An execution failure after a good quote counts as settled = 0 (100% shortfall, therefore severe).
2. recordFill (clean): fills++, cleanStreak++. If status is WATCH and the recovery condition holds, set NORMAL. If UNRATED and cleanStreak reaches 100, set NORMAL.
3. recordStrike: strikes++, cleanStreak = 0, lastStrikeAt = now, update cumShortfallBps. Reset the window if older than STRIKE_WINDOW, then recentStrikes++. If severe or recentStrikes reaches BLOCK_AFTER_STRIKES, set BLOCKED. Otherwise UNRATED and NORMAL become WATCH.
4. recordProbe: probes++. If shortfall exceeds the tolerance, treat as a strike (probeStrikes++, same rules). Otherwise cleanProbes++ and track distinct senders in a mapping; if cleanProbes and distinct senders reach the rating minimums and the hook is UNRATED, set NORMAL.
5. BLOCKED only leaves via reinstate (owner), which sets WATCH and resets the window.
6. Emit StatusChanged(hook, oldStatus, newStatus, reason) on every change, plus FillRecorded, StrikeRecorded and ProbeRecorded events carrying poolId and amounts.

8.4 FirmRouter

```
struct Route {
    PoolKey key; bool zeroForOne; bytes hookData;
    uint128 advertisedOut;   // NEW (P1, optional): quote-api's off-chain quote; 0 = unused
}
function swapExactIn(
    Route[] calldata routes,   // 1..MAX_LEGS (4), best quote first
    uint128 amountIn,
    uint128 minTotalOut,       // user's own floor across the whole swap
    address recipient,
    uint64 deadline
) external nonReentrant returns (uint256 amountOut, int256 legIndex);
// legIndex = filled route index, or -1 if nothing filled (input refunded)

event Filled(bytes32 indexed poolId, address indexed hook, address indexed user,
             uint256 amountIn, uint256 quotedOut, uint256 settledOut, uint8 legIndex);
event Rejected(bytes32 indexed poolId, address indexed hook, uint256 quotedOut,
               uint256 settledOut, bool execFailed);
event LegSkipped(bytes32 indexed poolId, address indexed hook, uint8 reason);
// reasons: 0 BLOCKED, 1 UNRATED_CAP, 2 QUOTE_FAILED, 3 BELOW_MIN_OUT,
//          4 ADVERTISED_MISMATCH (NEW)
event NoFill(address indexed user, uint256 amountIn);
error DeadlinePassed(); error BadRouteCount(); error RouteTokenMismatch();
error Shortfall(uint256 settledOut); error OnlySelf();
```

Algorithm (implement as written)

```
require(block.timestamp <= deadline); require(1 <= routes.length <= MAX_LEGS);
(tokenIn, tokenOut) = ends(routes[0]); all other routes must match, else RouteTokenMismatch
SafeERC20.safeTransferFrom(tokenIn, msg.sender, address(this), amountIn);
for i in 0 .. routes.length-1:
    hook = address(routes[i].key.hooks); pid = routes[i].key.toId();
    st = (hook == 0) ? NORMAL : scoreboard.statusOf(hook);
    if st == BLOCKED:                   emit LegSkipped(pid, hook, 0); continue;
    if st == UNRATED && amountIn > cap:  emit LegSkipped(pid, hook, 1); continue;
    quotedOut = _tryQuote(routes[i], amountIn);        // capped gas, 0 on any failure
    if quotedOut == 0:                  emit LegSkipped(pid, hook, 2); continue;
    tol   = (hook == 0) ? TOL_NORMAL : scoreboard.toleranceBps(hook);
    // NEW: off-chain advertised quote vs in-tx quote (no strike: could be market move)
    if routes[i].advertisedOut != 0 &&
       quotedOut < routes[i].advertisedOut * (10_000 - ADV_TOL) / 10_000:
                                        emit LegSkipped(pid, hook, 4); continue;
    floor = quotedOut * (10_000 - tol) / 10_000;
    if floor < minTotalOut:             emit LegSkipped(pid, hook, 3); continue;
    (ok, settled, execFailed) = _tryLeg(routes[i], amountIn, floor);
    if ok:
        safeTransfer(tokenOut, recipient, settled);
        if hook != 0: scoreboard.recordFill(hook, pid, quotedOut, settled);
        emit Filled(...); return (settled, i);
    else:
        if hook != 0: scoreboard.recordStrike(hook, pid, quotedOut, settled, execFailed);
        emit Rejected(pid, hook, quotedOut, settled, execFailed);
// no route filled
safeTransfer(tokenIn, msg.sender, amountIn); emit NoFill(msg.sender, amountIn);
return (0, -1);
```

[NEW] Why advertisedOut exists
Suppose a hook quotes high to off-chain eth_call (what quote-api and every aggregator sees) but quotes honestly in a real transaction. The in-transaction quote then equals the real fill, so no strike fires and the user simply receives the low price from the first route. Passing the off-chain quote and skipping any leg whose in-transaction quote falls short of it (default ADV_TOL = 50 bps) closes that gap, and the skip is logged for the scoreboard and the report. It cannot be a strike by itself because market movement between the off-chain read and the transaction is legitimate. **[A]** This is a v1.1 addition from design analysis; implement after the core path passes tests (P1).

Critical implementation details

- executeLeg(route, amountIn, floor) is external, require(msg.sender == address(this)). It calls the shared swap path and then if (out < floor) revert Shortfall(out). Because the comparison happens inside the reverting frame, the swap itself is undone while the router can still read the settled amount from the revert data.
- _tryLeg performs a low-level call to itself with {gas: LEG_GAS_CAP} and copies at most 68 bytes of return data with assembly, so a hostile hook cannot cause a return-data bomb. If it succeeds, decode settled. If it reverts with the Shortfall selector, decode settled. Any other failure (including out of gas) is execFailed = true and settled = 0.
- _tryQuote uses a capped-gas low-level call to the V4 Quoter (IV4Quoter.quoteExactInputSingle with a QuoteExactSingleParams of poolKey, zeroForOne, exactAmount and hookData) and returns 0 on failure. **[A]** Spike S1 confirms the Quoter works inside a transaction and measures its gas.
- Strikes must survive. Scoreboard writes happen in the outer frame after the leg's revert. When all legs fail the function must still return normally (refund and NoFill) rather than revert.
- Defaults: MAX_LEGS = 4; LEG_GAS_CAP and QUOTE_GAS_CAP start at 1,500,000 and are tuned from S1 measurements (Monad charges the gas limit, so lower caps save money). **[NEW]** Record measured gas per quote and per leg in docs/SPIKES.md and set caps to about 1.5x the measured worst honest case.
- Approvals: users approve FirmRouter directly with ERC-20 approve in the MVP. Permit2 support is optional.

Security checklist

- nonReentrant on swapExactIn; unlockCallback restricted to PoolManager; executeLeg restricted to self.
- Router must hold zero token balance after every call (assert in tests).
- No unbounded loops; fixed MAX_LEGS. No owner power over user funds; recovery function only for tokens sent by mistake and never for tokens mid-swap.
- Document that a hook can discriminate on the router's address and why the CanaryRunner uses other senders.

8.5 CanaryRunner and ProbeSender

```
// Permissionless. The caller supplies the dust and receives the output back.
function probe(Route calldata route, uint128 amountIn, uint8 senderIdx)
    external nonReentrant returns (uint256 quotedOut, uint256 settledOut);
event ProbeDone(bytes32 indexed poolId, address indexed hook, address indexed probeSender,
                uint128 amountIn, uint256 quotedOut, uint256 settledOut);
```

1. probeSender(senderIdx) is a deterministic ProbeSender contract (CREATE2 clone). Each index yields a different msg.sender seen by hooks. Deploy lazily.
2. Pull amountIn from the caller into the ProbeSender. The ProbeSender runs the same quote-then-execute path as the router (shared SwapExecutor and Quoter call) but does not revert on shortfall: it returns (quoted, settled) because the amount is dust.
3. The runner forwards the output to the caller and calls scoreboard.recordProbe(hook, poolId, probeSender, amountIn, quoted, settled).
4. If the quote fails, emit ProbeDone with quotedOut = 0 and do not change statistics.
5. Suggested probe matrix (driven by the agent): amounts {1, 10, 100} tokens x sender indices {0..3} x both directions, spread over several blocks, then re-probe every few hours to catch hooks that change behaviour later.

8.6 RawRouter (comparator)

swapExactInRaw(Route route, uint128 amountIn, uint128 minOut, address recipient) executes one route with no quote, no floor and no Scoreboard use, exactly like a naive aggregator that trusts the best quote. It exists for the side-by-side demo and the benchmark, and must be labelled 'unprotected comparator' in code and UI.

8.7 BondVault (P2, only after M7 is done)

- operators postBond(hook, token, amount); owner or governance certify(hook). Certified hooks get a CERTIFIED flag on the Scoreboard view.
- When a certified hook delivers less than quoted but within tolerance, FirmRouter calls coverShortfall(hook, user, token, shortfall) so the user still receives exactly the quote.

- On a rejected leg from a certified hook the vault slashes a fixed percentage of the bond to a treasury address.
- Withdrawals are delayed (for example 24 h) and blocked while the hook is WATCH or BLOCKED.

8.8 Test fixtures (quarantined, testnet only)

Safety note
These contracts intentionally misbehave so the defence can be measured. Keep them in contracts/test-fixtures, never deploy them to mainnet, never submit them to any hook registry, and label them clearly in the README. Describe-and-implement behaviours only; do not publish tricks for evading detection.

Fixture	Behaviour to implement	What it proves
FixtureHonestHook	Takes a constant, disclosed fee identically on quote and real paths	No false positives
FixtureContextHook	Gives the quoted amount on the quote path but takes an extra fee (target about 18%) on the real execution path. **[NEW]** Key the difference on the swap caller (Quoter vs router), which matches real quote-spoofing and is what the in-tx Quoter exposes	Same-transaction check catches it; strike and fallback work
FixtureRandomFeeHook	Takes a fee of 5% to 20% on a pseudo-random subset of real swaps	Probabilistic spoofing is caught over repeated probes
FixtureSenderAwareHook	Honest toward the FirmRouter address (set in constructor), takes a fee for every other caller	Why CanaryRunner uses other senders; scoreboard marks it from probes
FixtureGasGriefHook	Consumes all available gas on the real path	LEG_GAS_CAP and fallback work
FixtureRevertHook	Quotes fine, then reverts on the real path	Execution failure is treated as a severe strike
[NEW] FixtureQuoteOnlyHook (P1)	Quotes high only when the caller is the Quoter outside a transaction context; honest in a real transaction	Tests the advertisedOut skip (reason 4)
MockToken	ERC-20, 18 decimals, public mint capped per call (tUSD and tUSDC)	Funded testnet liquidity and demo balances

Hook addresses must encode the permission flags the fixture needs (for example beforeSwap, afterSwap and the swap-delta-return flags). Mine them with the example repo's NamespacedHookMiner and the CREATE2 factory. **[A]** Confirm flag bit positions in v4-core Hooks.sol (expected layout in 5.5).

8.9 Pools to seed on testnet

Pool	Hook	Purpose
A	None	Baseline route; always allowed
B	FixtureContextHook	Configured so it quotes the BEST price; the demo's villain
C	FixtureHonestHook	Honest alternative with a slightly lower quote than B
D	FixtureRandomFeeHook	Probabilistic spoofing
E	FixtureSenderAwareHook	Shows canary value (honest to FirmRouter only)

Use the tUSD/tUSDC pair, equal wide liquidity around 1:1 through PositionManager (follow the example's PoolLifecycle.s.sol). Choose fees so the quote order is B, C, A for a 1,000-token trade.

8.10 Contract tests (must exist and pass)

- Scoreboard: every transition in 8.3 including edge cases; reporter allow-list; reinstate; parameter bounds.
- FirmRouter: honest fill; context-hook rejection with fallback to the next route; all legs fail then refund with strikes persisted; BLOCKED skip; UNRATED cap; quote failure skip; gas-grief hook; revert hook; return-data bomb; minTotalOut; deadline; token mismatch; reentrancy attempt; **[NEW]** advertisedOut mismatch skip.

- Invariants: router token balance is zero after every call; the user receives at least the floor of the filled leg or gets the full input back; a hook's strikes never decrease.
- Fuzz: amounts, tolerances, shortfall sizes, route orderings.
- CanaryRunner: distinct sender addresses per index; clean and strike probes drive status as specified; failed quote leaves stats unchanged.
- Fork test: run the core flow against a fork of testnet using the real PoolManager and Quoter.

8.11 Deployment scripts

- Deploy.s.sol: deploy Scoreboard, FirmRouter, CanaryRunner, RawRouter; wire reporters; write addresses, block numbers and tx hashes to deployments/10143.json.
- DeployFixtures.s.sol and SeedPools.s.sol: deploy MockTokens and fixture hooks (mined salts), initialise pools and add liquidity.
- Redeploy.sh: one command that rebuilds everything from scratch, because the testnet can reset.
- Verify every contract on the explorer following the verification step in the Monad hooks Deploy guide. Record verification status in DEPLOYMENTS.md.
- Set explicit gas limits in scripts (gas is charged on the limit).

9. Indexer (Envio HyperIndex)

Purpose: give the web app, quote-api and agents a fast GraphQL view of pools, fills, strikes, probes and hook status. Envio lists Monad Testnet as first-class support for HyperIndex, HyperSync and HyperRPC and has a free tier **[V]**. **[NEW]** Use the endpoints in 5.4 (monad-testnet.hypersync.xyz), follow Envio's getting-started guide and let envio init generate the chain entry. If the indexer costs more than half a day, switch to fallback F4.

9.1 Sources to index

Contract	Events	Why
Uniswap v4 PoolManager (0x451D...643d)	Initialize(id, currency0, currency1, fee, tickSpacing, hooks, sqrtPriceX96, tick) **[A]** confirm in IPoolManager.sol	Discover pools and their hooks
Scoreboard	FillRecorded, StrikeRecorded, ProbeRecorded, StatusChanged	Hook statistics and status history
FirmRouter	Filled, Rejected, LegSkipped, NoFill	Swap history and the swap timeline
CanaryRunner	ProbeDone	Probe grid per hook

9.2 Schema (schema.graphql)

```
type Hook {
  id: ID!                  # lowercase hook address (0x0 for hookless is not stored)
  status: String!          # UNRATED | NORMAL | WATCH | BLOCKED
  fills: Int! strikes: Int! probes: Int! probeStrikes: Int!
  avgShortfallBps: BigInt! # cumShortfall / max(strikes, 1)
  lastStrikeAt: Int
  flags: [String!]!        # decoded permission flags (computed in handler from address)
  pools: [Pool!]! @derivedFrom(field: "hook")
}
type Pool { id: ID! hook: Hook currency0: String! currency1: String!
            fee: Int! tickSpacing: Int! createdAt: Int! }
type Fill { id: ID! pool: Pool! hook: Hook user: String! amountIn: BigInt!
            quotedOut: BigInt! settledOut: BigInt! legIndex: Int! txHash: String! ts: Int! }
type Strike { id: ID! pool: Pool! hook: Hook! quotedOut: BigInt! settledOut: BigInt!
              shortfallBps: Int! execFailed: Boolean! txHash: String! ts: Int! }
type Probe { id: ID! pool: Pool! hook: Hook! probeSender: String! amountIn: BigInt!
             quotedOut: BigInt! settledOut: BigInt! shortfallBps: Int! ts: Int! }
type StatusChange { id: ID! hook: Hook! oldStatus: String! newStatus: String!
                    reason: String! ts: Int! }
type Skip { id: ID! pool: Pool! hook: Hook reason: Int! txHash: String! ts: Int! }  # NEW
```

9.3 Handler rules

- Decode hook permission flags from the hook address low bits in the Initialize handler **[A]** using the layout in 5.5. Store human-readable names such as BEFORE_SWAP, AFTER_SWAP, BEFORE_SWAP_RETURNS_DELTA, AFTER_SWAP_RETURNS_DELTA.
- Create the Hook entity lazily and keep status in sync with StatusChanged events; do not recompute status offchain.
- Compute shortfallBps in the handler with the same formula as the contract.
- Use deployments/10143.json for addresses and start blocks so redeploys are one config change. Provide a script that regenerates config.yaml from that file.
- Expose the hosted GraphQL endpoint to the web app and services through env var ENVIO_GRAPHQL_URL.

9.4 Example queries to support

```
# leaderboard
{ Hook(order_by: {strikes: desc}, limit: 50) {
    id status fills strikes probes avgShortfallBps lastStrikeAt flags } }
# swap timeline / recent activity
{ Strike(order_by: {ts: desc}, limit: 20) { hook { id } shortfallBps quotedOut settledOut txHash ts } }
# probe grid for a hook
{ Probe(where: {hook_id: {_eq: "0x..."}}, order_by: {ts: desc}) {
    probeSender amountIn quotedOut settledOut shortfallBps ts } }
```

10. quote-api service

TypeScript, Hono, viem. Stateless except a small in-memory cache. It orders candidate routes; it is never trusted for safety because FirmRouter re-quotes and re-checks onchain.

10.1 Endpoints

Method and path	Request	Response
GET /health	none	{ ok, chainId, blockNumber, rpcInUse }
GET /pools	tokenIn, tokenOut	Pools for the pair with hook, status, fee, liquidity
POST /route	{ tokenIn, tokenOut, amountIn, maxLegs? }	{ routes: [{ key, zeroForOne, hookData, quotedOut (advertisedOut), status, floor }], skipped: [...] } ordered best quote first
GET /hooks	optional status filter	Hook list with statistics (proxies the indexer)
GET /hooks/:addr	hook address	Hook stats, flags, probe grid, latest report
GET /hooks/:addr/report	hook address	Latest report (markdown) plus the structured evidence it was based on

10.2 Route algorithm

1. Query the indexer for pools whose currencies match the pair. Drop pools with zero liquidity using StateView.getLiquidity (or the matching call confirmed from the deployed ABI).
2. Read hook status from the Scoreboard in one Multicall3 batch. Drop BLOCKED hooks and list them under skipped with a reason. Drop UNRATED hooks when amountIn exceeds the cap.
3. Batch V4 Quoter calls (quoteExactInputSingle) through Multicall3 using eth_call. Discard failures.
4. Sort by quotedOut descending, take maxLegs (default 4, never more than the contract's MAX_LEGS), compute floor = quotedOut * (10000 - tolerance) / 10000 for display. Return quotedOut as each route's advertisedOut.
5. Return the routes plus a Protected-versus-Raw preview: the naive pick is the first route; the FIRM pick is simulated with eth_call on FirmRouter.swapExactIn when a wallet address is supplied.

10.3 Reliability rules

- RPC pool: primary testnet-rpc.monad.xyz, then Ankr, then the Monad Foundation RPC, then optional Alchemy. Round-robin with health checks, honouring each endpoint's rate limit (25 req/s for eth_call on the primary; the Foundation RPC does not allow batching, so route Multicall3 batches elsewhere).
- Cache quotes and pool lists for 1 to 2 seconds. Deduplicate concurrent identical requests.
- Validate all inputs with zod. Return typed errors. Log structured JSON. No private keys in this service.
- Unit tests with a mocked RPC and an integration test against testnet using the seeded pools.

11. Agents

Design rule
Agents surface problems and use the product. The enforcement is deterministic contract logic. The LLM never decides whether a swap executes or what status a hook gets. Every agent action that touches funds uses testnet keys only.

11.1 canary-agent (most important agent)

Problem it solves: hooks that detect simulation cannot be caught by eth_call-style quotes, and users should not be the first test subjects. The canary tests new hooks with small real swaps before users arrive. **Under the 7-day plan, build the CanaryRunner contract and a minimal script-driven prober; the LLM report and SQLite scheduler are optional.**

1. **Discover:** subscribe to the indexer (or poll PoolManager Initialize events) for new pools with a non-zero hook and liquidity.
2. **Static analysis (deterministic):** decode permission flags from the hook address. Fetch deployed bytecode with eth_getCode and run a small disassembler (skip PUSH data) to list context-sensitive opcodes: ORIGIN, GAS, GASPRICE, PREVRANDAO, NUMBER, TIMESTAMP, COINBASE, BLOCKHASH, BALANCE, SELFBALANCE, EXTCODESIZE and EXTCODEHASH. Output riskFlags[]. These flags only prioritise probing and enrich the report; many honest hooks use some of them, so flags never change status.
3. **Probe (onchain evidence):** call CanaryRunner.probe for the matrix amounts {1, 10, 100} x sender indices {0..3} x both directions, spread across several blocks. Dust is free on testnet. The agent wallet supplies the tokens (mint from MockToken).
4. **Re-probe:** schedule a re-run every few hours and after any hook-owner upgrade event, to catch hooks that change behaviour after being rated.
5. **Report (LLM, advisory):** feed the structured evidence JSON (flags, probe table, current status) to the LLM and ask for a plain-language summary of at most 150 words. Store it in SQLite and serve it via quote-api. Sanitize output; never execute or parse it for decisions.

- Wallet: a dedicated testnet key from .env, funded from the faucet, with its own tUSD/tUSDC balances.
- LLM client: OpenAI-compatible chat completions with env vars LLM_BASE_URL, LLM_API_KEY, LLM_MODEL so Kimi, Qwen or others can be swapped in.
- Idempotency: persist probe schedules and results so restarts do not double-probe. Use a simple SQLite file.
- Output the probe grid in the same shape the UI renders (sender x amount with shortfall bps).
- Tests: static-analysis unit tests using fixture bytecode; an integration test that probes each fixture hook and asserts the resulting status.

11.2 firm-mcp and trader-agent (the AI customer) - P1/P2 under the 7-day plan

Problem it solves: autonomous trading agents act on quotes without human review, so they are the most exposed users. The demo shows an agent that is naive but cannot lose money because protection lives in the contract, not in the prompt. **If time is short, replace the LLM agent with a scripted trader that picks the best quote; the benchmark result (B8) is the same and the fallback F6 already allows it.**

Tool	Inputs	Returns	Notes
list_pools	tokenIn, tokenOut	pools with hook, status, quotedOut	Reads quote-api
get_hook_score	hook	status, fills, strikes, avg shortfall, latest report	Reads quote-api
quote	tokenIn, tokenOut, amountIn	ordered routes with floors	Reads quote-api
firm_swap	tokenIn, tokenOut, amountIn, minOut	tx hash, filled leg, settled amount, rejected legs	Calls FirmRouter.swapExactIn

Tool	Inputs	Returns	Notes
raw_swap	poolId, amountIn	tx hash, settled amount	Calls RawRouter. Benchmark mode only, label as unprotected

1. System prompt: 'You are a trading agent. Swap the stated amount of tUSD to tUSDC at the best available price. Use the tools. Choose the pool with the best quoted output.' The honest-but-naive behaviour is the point; no prompt injection is needed.
2. Run A (unprotected): the agent has list_pools, quote and raw_swap only. It picks the best quote (Pool B) and loses about 18%. Record settled versus quoted.
3. Run B (protected): the agent has list_pools, quote and firm_swap. It still prefers Pool B by quote, but FIRM rejects it and fills elsewhere. Record the result.
4. Repeat N times (for example 20) with varied amounts. Save results to a JSON file that the Arena page and bench/read.

11.3 receipt-writer

Turns each Filled or Rejected event into one readable line, for example: 'Pool B quoted 999.0 but delivered 819.2 (-18.0%). Rejected and struck. Filled via Pool C at 998.2.' Start with a deterministic template. Offer an optional LLM rewrite flag. Used by the Swap timeline and the hook detail page.

11.4 Optional P2 agent: MetaMask Agent Wallet plugin

MetaMask Agent Wallet plugins are npm packages (oclif) that declare capabilities such as wallet-read and wallet-submit. A plugin command like mm firm swap could route an agent's swap through FIRM. Access requires MetaMask's Early Access Program, so treat it as a stretch goal and verify access before spending time.

12. Frontend (apps/web)

Next.js App Router, React, Tailwind CSS, wagmi v2 and viem for wallet and contract calls, TanStack Query for data, graphql-request (or urql) for the Envio endpoint. Dark theme with clear red (BLOCKED), amber (WATCH), grey (UNRATED) and green (NORMAL) status colours. Desktop-first but responsive.

12.1 Chain and data setup

- Define Monad Testnet as a custom chain: id 10143, name Monad Testnet, native currency MON (18), RPC https://testnet-rpc.monad.xyz, explorer https://testnet.monadvision.com.
- Load addresses and ABIs from deployments/10143.json and services/shared/abis. Never hard-code addresses in components.
- Env: NEXT_PUBLIC_QUOTE_API_URL, NEXT_PUBLIC_ENVIO_GRAPHQL_URL, NEXT_PUBLIC_WALLETCONNECT_ID (if used).
- Show a persistent banner: 'Monad Testnet. Test tokens only.' with links to the faucet and a Mint test tokens button (MockToken.mint).

12.2 Screens

Route	Purpose	Key UI and behaviour	Plan
/	Landing	One-sentence pitch, the 18% example, a Try the demo button, and the 0x evidence with proper attribution	P0
/swap	Protected versus Raw swap	Token pair and amount; route preview from quote-api (quote, floor, hook badge, status); toggle Protected (FirmRouter) or Raw (RawRouter, labelled unprotected); after sending, a live timeline built from receipt logs (Skipped, Trying, Rejected with numbers, Filled)	P0
/leaderboard	Public scoreboard	Table of hooks sorted by status then strikes: status chip, fills, strikes, probes, average shortfall, last strike time. Auto-refresh every few seconds	P0
/hooks/[address]	Hook detail	Decoded flags, static-analysis risk flags, probe grid (sender x amount, shortfall bps), strike history with explorer links, latest report, status history	P1
/arena	AI agent comparison	Two columns (Unprotected agent versus Protected agent) with a running loss counter, per-run table and summary stats from the scenario runner results	P1/P2
/integrate	For other routers	Short code sample of reading Scoreboard.statusOf(hook) and toleranceBps(hook), plus the deployed addresses and ABIs	P0

12.3 Components to build

- StatusChip, HookBadge, ShortfallBar (visual quote-versus-settled bar), RouteCard, TimelineStep, ProbeGrid, StrikeRow, LossCounter, CodeSnippet, TxLink.
- useRoutes (quote-api), useHooks and useHook(addr) (indexer), useSwap (simulate then write FirmRouter or RawRouter), useReceiptTimeline (parse logs).
- Handle ERC-20 approval as a visible first step. Show clear errors for rejected transactions and for the NoFill case (input returned, strikes recorded).
- Before sending, run simulateContract on swapExactIn and display the predicted outcome. Set an explicit gas limit (Monad charges the limit).

12.4 Swap timeline logic

```
parse receipt logs in order:
  LegSkipped(poolId, hook, reason)
     -> "Skipped Pool X (hook 0x.. is BLOCKED)" and similar by reason code (0-4)
  Rejected(poolId, hook, quoted, settled, execFailed)
     -> "Pool X quoted 999.0, delivered 819.2 (-18.0%). Rejected, strike recorded."
  Filled(poolId, hook, user, amountIn, quoted, settled, legIndex)
     -> "Filled via Pool Y: received 998.2 (matches quote)."
  NoFill(user, amountIn)
     -> "No pool honoured its quote. Your tokens were returned."
```

12.5 Optional P2 UI features

- Passkey login: Privy supports passkeys and is reported to work on Monad testnet; Mera is the Monad Foundation option. Read the docs and confirm testnet use before integrating.
- A rate-limited demo relayer so judges can try the swap without funding a wallet.
- Share-card image of a prevented loss.

12.6 Quality bar

- Lighthouse accessibility reasonable; keyboard focus visible; colour is never the only status signal (also show text).
- Empty, loading and error states for every query. No console errors in the demo path.
- Playwright smoke test: load /swap, mock quote-api, run a protected swap against the testnet seeded pools.

13. Testing and benchmark plan

13.1 Test layers

Layer	Tooling	What it covers
Contract unit, fuzz, invariant	forge test	Section 8.10 list
Contract fork	forge test --fork-url (testnet RPC)	Real PoolManager, Quoter and seeded pools
Service unit	vitest	quote-api route logic with mocked RPC; static analysis with fixture bytecode; scenario runner parsing
Integration	vitest + testnet	quote-api against seeded pools; canary-agent probing each fixture hook
End to end	Playwright + scripted run	Swap page protected flow; leaderboard updates after a strike

13.2 Benchmark design (bench/ writes RESULTS.md)

The benchmark is the project's proof. It must run from one command (pnpm bench) against testnet and regenerate RESULTS.md with a timestamp and the deployment addresses used. Under the 7-day plan, B1, B2, B3, B5 and B6 are P0; B4 and B7 are P1; B8 is P2.

Exp.	Name	Method	Metric
B1	Detection by hook	For each fixture hook: 12 probes (3 amounts x 4 senders) plus 20 FirmRouter swaps	Catch rate: strikes divided by deviating executions; time or probes to BLOCKED/WATCH
B2	False positives	FixtureHonestHook: at least 50 probes and fills	Number of strikes (target 0) and final status (target NORMAL)
B3	User loss	20 swaps of 1,000 tUSD each through RawRouter and through FirmRouter, with Pool B quoting best	Loss versus quote in bps and in tokens; loss versus the best honest alternative
B4	Sender-aware hook	Probes through FirmRouter only versus probes through CanaryRunner senders	Detection with and without canaries (shows why the canary matters)
B5	Gas overhead	Same swap through RawRouter and FirmRouter (honest pool). **[NEW]** Also record V4Quoter gas per leg	Gas used and gas limit set (Monad charges the limit); report the ratio honestly
B6	Resilience	Gas-grief and revert fixtures	Router never reverts as a whole; input is refunded or filled elsewhere
B7	Dashboard latency	Time from tx confirmation to the event appearing in GraphQL	p50 and p95 seconds
B8	Agent arena	trader-agent runs A and B, N = 20	Total loss with and without FIRM; success rate

Formulas

```
shortfall_bps       = max(0, quoted - settled) * 10000 / quoted
loss_vs_quote_bps   = shortfall_bps of the leg that actually paid the user
loss_vs_best_honest = (best honest quote - received) / best honest quote * 10000
catch_rate          = strikes_recorded / deviating_executions_attempted
false_positive_rate = strikes_on_honest_hook / total_honest_executions
```

Expected outcomes to sanity check against: FixtureContextHook should be caught every time; FixtureRandomFeeHook is caught with near certainty after about a dozen probes; FixtureHonestHook should show zero strikes; FixtureSenderAwareHook should look clean through FirmRouter but be flagged by canary probes. Report whatever you actually measure, including failures. **[NEW]** State in RESULTS.md that fixtures are our own, so the catch rate shows the mechanism works, not that the field is covered.

14. Configuration and operations

14.1 Environment variables (see Appendix B)

Variable	Used by	Notes
DEPLOYER_ACCOUNT (Foundry keystore) or DEPLOYER_PRIVATE_KEY	contracts scripts	Testnet only. Prefer cast wallet import so the key is never in a file
MONAD_TESTNET_RPC_URLS	all services	Comma-separated, primary first
ALCHEMY_MONAD_TESTNET_URL	optional	Fallback RPC if a key is available
ENVIO_GRAPHQL_URL	web, quote-api, agents	Hosted indexer endpoint
QUOTE_API_URL	web, agents	Base URL of quote-api
CANARY_PRIVATE_KEY	canary-agent	Dedicated testnet key
TRADER_PRIVATE_KEY	trader-agent, firm-mcp	Dedicated testnet key
LLM_BASE_URL, LLM_API_KEY, LLM_MODEL	canary-agent, trader-agent	Any OpenAI-compatible endpoint
SQLITE_PATH	canary-agent	Probe schedule and reports

14.2 Hosting suggestions

- Web app and quote-api: Vercel (or any Node host). Canary-agent and firm-mcp: a small always-on host such as Railway or Fly. Indexer: Envio hosted service.
- Keep keys in the host's secret store. Use separate funded keys for deployer, canary and trader.

14.3 Redeploying after a testnet reset

1. Run contracts/script/Redeploy.sh (deploys contracts, fixtures and seeded pools and rewrites deployments/10143.json).
2. Run the indexer config regeneration script, then redeploy the indexer.
3. Restart services (they read deployments/10143.json). Re-run the canary and benchmark. Update DEPLOYMENTS.md.

15. Spikes and fallbacks

Run these in milestone M0 and record outcomes in docs/SPIKES.md (question, test, result, decision). Each has a pre-agreed fallback so work never stalls. Spend at most half a day on M0 in total under the 7-day plan.

Spike	Question	Pass criteria	Fallback if it fails
S1 Quoter in transaction	Can a contract call IV4Quoter.quoteExactInputSingle inside a transaction, and at acceptable gas on Monad? [NEW] Uniswap documents the Quoter as not for onchain use	Returns a non-zero amount that matches the real output of the same swap on an honest pool (within 1 wei) in a fork test. [NEW] Gas per quote is measured and one quote plus one leg fits a per-leg budget you can state (record in SPIKES.md); worst-case 4-leg limit still under the block gas limit	F1: the caller passes quotedOut computed off-chain with a freshness limit (a few blocks) and a wider tolerance. Document as weaker because market movement can no longer be separated from spoofing. Do not substitute a self-simulation through FirmRouter (see 4.2)
S2 Hook mining and swap	Can we mine a hook address with the needed permission bits, initialise a pool and swap through it?	A swap through FixtureHonestHook succeeds and the hook callbacks run	F2: start from the example repo's Counter hook and the pinned dependencies; recheck the bit layout in Hooks.sol
S3 Isolated leg	Do try/catch with a gas cap and a self-call isolate a failing leg while strikes persist, and is returndata capped?	A reverting or gas-griefing fixture leaves the user's funds intact, the strike recorded and the next route filled	F3: move the leg execution into a separate LegExecutor contract called with try/catch, giving the same isolation through a different call frame
S4 Event shapes	What are the exact Initialize event signature and hook flag bit positions?	Indexer handler decodes a real Initialize event from testnet; mined fixture addresses match the expected bits	Read v4-core IPoolManager.sol and Hooks.sol; adjust schema and handlers

Other fallbacks

Risk	Fallback
Envio indexing problems	F4: a small viem getLogs poller writing to SQLite with the same GraphQL-like endpoint shape
RPC rate limits hit	F5: aggressive caching, batching through Multicall3 (not on the Foundation RPC), more endpoints in rotation
LLM provider unavailable	F6: deterministic template reports and a scripted trader that reproduces recorded runs
Testnet reset	F7: Redeploy.sh and regenerated indexer config (Section 14.3)
Pools do not seed as planned	F8: simplify to hookless pool A plus pools B and C only; keep D and E as stretch
[NEW] Out of time	F9: ship M0 to M4 + minimal M7 + M8 (see 16.1). A working protected-vs-raw swap with measured results beats a wide, unfinished build

16. Milestones, acceptance criteria and prompts

Work strictly in order. P0 means required for a credible submission, P1 means strongly desired, P2 means only if time remains. If time runs short, cut from the end of P1 and skip P2, but never skip M7 and M8.

16.1 [NEW] 7-day plan (6 to 13 Oct 2026)

Proposed compression of M0 to M8. Today is 6 Oct and the deadline is 13 Oct, 11:59 PM ET. Aim to submit by 12 Oct. The human may change this plan; Claude Code should follow it unless told otherwise.

Day	Date	Work	Exit check
1	Tue 6 Oct	M0: repo, Foundry 1.8.0, reproduce the Monad example swap, spikes S1 to S4 (half a day), keys and faucet funds	docs/SPIKES.md has S1 to S3 results and fallbacks chosen
2	Wed 7 Oct	M1: SwapExecutor, Scoreboard (all transitions), FirmRouter, RawRouter and their tests	forge test green; router balance zero invariant
3	Thu 8 Oct	M2: fixtures, mined hook addresses, pools B, C, A (D and E only if easy), deploy, verify, DEPLOYMENTS.md	Scripted swap rejects B, fills C, strike recorded on testnet
4	Fri 9 Oct	M3: indexer (or F4 poller) and M4 part 1: quote-api	Example queries return data after a swap; /route returns B, C, A order
5	Sat 10 Oct	M4 part 2: web /, /swap, /leaderboard, /integrate; minimal CanaryRunner contract and script (M5 core)	Raw vs protected swap works in the browser
6	Sun 11 Oct	M7: bench B1, B2, B3, B5, B6 (B4 if the canary works); RESULTS.md and THREAT_MODEL.md	pnpm bench regenerates RESULTS.md
7	Mon 12 Oct	M8: README (with AI-tool disclosure and Why Monad), demo video, fresh-clone test, submit with margin	Checklist in 17.1 fully ticked
Buffer	Tue 13 Oct	Only fixes and a re-submit if needed. Version at the deadline is judged	Submitted

Deferred under this plan: hook detail page polish, LLM report, firm-mcp and trader-agent (use the scripted trader for B8 if time allows), Arena, advertisedOut (P1; do it only if M1 finishes early), BondVault, Mera or Privy passkeys, mainnet scan, MetaMask plugin.

M0 (P0): Setup and spikes

Goal: Prove the riskiest assumptions before building on them.

- Create the repo with the Section 7 layout, MIT licence, CLAUDE.md (Appendix A) and .env.example.
- Install Foundry v1.8.0. Clone the Monad hooks example. Fund a testnet account. Run the example's lifecycle and confirm a swap through Universal Router.
- Implement spikes S1 to S4 as Forge tests or scripts. Write results and decisions to docs/SPIKES.md.

Done when: S1 to S3 pass or a fallback is chosen and recorded, and the example swap works on testnet.

> **Prompt for Claude Code**
> Read sections 0, 4, 5 and 8 of the FIRM spec. Create the repository layout from section 7. Install Foundry v1.8.0, reproduce the Monad Uniswap v4 hooks example on testnet, then implement spikes S1 to S4 from section 15 and record outcomes in docs/SPIKES.md. Include Quoter gas measurements. Stop and report if any spike fails and propose the pre-agreed fallback.

M1 (P0): Core contracts

Goal: Implement the enforcement logic with full tests.

- SwapExecutor, Scoreboard (all transitions), FirmRouter and RawRouter per Section 8.
- Unit, fuzz and invariant tests from 8.10 using mocks or fixtures as needed.

Done when: forge test passes, the router holds zero balance after each call, and every Scoreboard transition has a test.

Prompt for Claude Code
Implement SwapExecutor, Scoreboard, FirmRouter and RawRouter exactly as specified in section 8, including the strike-persistence behaviour and capped return data. Write the tests listed in 8.10. Do not deploy yet. Report any deviation from the spec and why.

M2 (P0): Fixtures, tokens, pools and deployment

Goal: Get everything live on Monad Testnet.

- Quarantined fixture hooks and MockToken (Section 8.8), mined hook addresses, seeded pools A to E (8.9).
- Deploy.s.sol, DeployFixtures.s.sol, SeedPools.s.sol, Redeploy.sh; verify contracts; write deployments/10143.json and DEPLOYMENTS.md.

Done when: A script-driven swap through FirmRouter on testnet rejects Pool B, fills elsewhere and records a strike; addresses and tx hashes are documented.

Prompt for Claude Code
Implement the quarantined fixtures and mock tokens, mine hook addresses, seed pools so the quote order is B, C, A, deploy all contracts to Monad Testnet (10143) with explicit gas limits, verify them, and write deployments/10143.json and DEPLOYMENTS.md. Then run a scripted protected swap and show the Rejected and Filled events.

M3 (P0): Indexer

Goal: Make the history queryable.

- Envio HyperIndex project per Section 9 with a config generated from deployments/10143.json.
- Deploy to Envio hosted; document the GraphQL endpoint.

Done when: The three example queries in 9.4 return live data after a protected swap.

Prompt for Claude Code
Build the Envio HyperIndex indexer described in section 9 for Monad Testnet using deployments/10143.json. Confirm the Initialize event signature from v4-core before writing the handler. Deploy it and verify the example queries return data after a swap. If it takes more than half a day, switch to fallback F4.

M4 (P0): quote-api and the Swap and Leaderboard screens

Goal: End to end protected-versus-raw demo in the browser.

- quote-api per Section 10 with RPC pooling and caching.
- Web app: chain setup, mint tokens, /swap with timeline, /leaderboard, /integrate.

Done when: A user can run the same swap through RawRouter and FirmRouter in the UI and see the loss versus the rejection, with the leaderboard updating.

Prompt for Claude Code
Implement services/quote-api and apps/web per sections 10 and 12 for the /, /swap, /leaderboard and /integrate routes. Use deployments/10143.json for addresses, simulate before sending, and render the receipt timeline from logs. Add the Playwright smoke test.

M5 (P1): CanaryRunner, canary-agent and hook detail page

Goal: Detect suspect hooks before users do.

- CanaryRunner and ProbeSender with tests (Section 8.5).
- canary-agent per Section 11.1 (minimal under the 7-day plan).
- /hooks/[address] page with the probe grid and report.

Done when: Deploying a new fixture hook pool leads the agent to probe it and the hook reaches WATCH or BLOCKED (or NORMAL for the honest hook) without any user trade; the sender-aware hook is flagged only by canary probes.

Prompt for Claude Code
Implement CanaryRunner and ProbeSender (section 8.5), then services/canary-agent (section 11.1) and the /hooks/[address] page. Keep deterministic logic for status and use the LLM only for the report. Add tests including one that probes every fixture hook and asserts final statuses.

M6 (P2 under the 7-day plan): firm-mcp, trader-agent and Arena

Goal: Show an AI agent that cannot be cheated.

- firm-mcp tools (Section 11.2) and the scenario runner for runs A and B, N = 20.
- /arena page reading the saved results.

Done when: The Arena shows total loss for the unprotected agent versus the protected agent from real testnet runs.

Prompt for Claude Code
Implement services/firm-mcp and services/trader-agent per section 11.2 using an OpenAI-compatible LLM endpoint from env vars. Run scenarios A and B 20 times each against testnet, save JSON results, and build the /arena page from them.

M7 (P0): Benchmark and threat model

Goal: Produce the evidence.

- bench/ implementing B1 to B8 (Section 13.2) writing RESULTS.md.
- THREAT_MODEL.md: assets, actors, attacks, mitigations, residual risks, trusted roles, the sender-aware limitation.

Done when: pnpm bench regenerates RESULTS.md with real numbers, including any unfavourable ones.

Prompt for Claude Code
Implement bench/ for experiments B1 to B8 (P0 subset in 13.2), run it against testnet, and generate RESULTS.md with the formulas from section 13.2. Write THREAT_MODEL.md honestly, including limits. Do not round numbers in our favour.

M8 (P0): Polish and submission

Goal: Make it judge-ready.

- README, architecture diagram, setup and redeploy instructions, 'Why Monad' paragraph, AI-tool disclosure, pre-existing code disclosure.
- Demo script and recording plan (Section 17). Final check of every link and address.

Done when: Fresh clone plus documented steps reproduces the demo; submission checklist (Section 17) is fully ticked.

Prompt for Claude Code
Finalise README.md (including the AI-tool disclosure), DEPLOYMENTS.md, RESULTS.md and THREAT_MODEL.md, verify a fresh clone works following the README, and prepare the submission checklist in section 17.

P2 stretch list (only after M7)

- BondVault with certification and shortfall repayment (Section 8.7).
- Passkey login (Privy or Mera; verify testnet behaviour first).
- Demo relayer so judges can try the swap without MON.
- Read-only mainnet scan: use the Uniswap hook registry for Monad and Envio HyperSync on chain 143 to count hooks with swap-delta flags and publish a 'state of Monad hooks' chart. Present flags as risk indicators, never as accusations against named hooks.
- MetaMask Agent Wallet plugin (needs Early Access).

17. Submission checklist and demo script

17.1 Checklist

- Public GitHub repository with an OSI licence (MIT). No secrets in history. Commit history spans the build window.
- README: problem, solution, architecture diagram, quick start, redeploy, limits, links. [NEW] **Includes a statement that AI coding tools (Claude Code) were used.**
- DEPLOYMENTS.md: every contract address, deployment tx hash and verification status on Monad Testnet; indexer and service URLs.
- RESULTS.md: benchmark tables with the exact commit and deployment used.
- THREAT_MODEL.md and docs/SPIKES.md.
- Live demo URL and a 60 to 90 second video, plus a longer walkthrough if the platform allows.
- 'Why Monad' paragraph (draft below) included in the README and the submission write-up.
- Disclose any pre-existing code or libraries. Everything shown should have been built during the hackathon window.
- Team registered on hackathon.monad.xyz (1 to 5 members, a primary contact designated for prizes). [NEW] One project, one track (Track 1); multiple submissions disqualify.
- Re-read the official rules modal on hackathon.monad.xyz (open-source requirement, testnet acceptance, eligibility, AI disclosure) and submit by **12 Oct**, leaving a day of margin before the 13 Oct 11:59 PM ET deadline (confirm the exact time and timezone on the platform).
- Tick the sponsor bounties actually used and whose rules you have read (Envio, LLM credits such as Kimi or Alibaba Cloud, Privy or Mera if integrated).

17.2 Draft: Why Monad

FIRM compares a pool's quote with its real fill inside a single transaction. Monad's fast block production and finality make that protection feel instant to users, and its low fees make it affordable to write a fill or strike record for every swap and to run dust probes continuously. Uniswap v4, whose hooks create the risk we address, is already deployed on Monad, and Monad's ecosystem has publicly discussed quote spoofing as a hidden cost to traders. We deployed everything to Monad Testnet and index it with Envio.

[NEW] Be accurate: Monad charges gas on the gas limit, so state the measured gas overhead from B5 alongside the 'low fees' claim.

17.3 Demo script (60 to 90 seconds)

Time	Screen	Narration
0:00	Landing	Routers rank pools by their quote. Hooks can lie about it. 0x classed 54.2% of the v4 hooks it examined as malicious, and 26.4% more as likely malicious.
0:10	/swap, Raw mode	Same trade, naive router. Best quote 999. We get 819. 180 tokens gone.
0:25	/swap, Protected mode	Now FIRM. It quotes, executes, sees the shortfall, reverts that leg and tries the next pool.
0:40	Timeline and /leaderboard	We got 998.2. And the hook is now publicly red, for every router.
0:50	/hooks/[address]	Before users arrive, our canary probes new hooks with dust swaps from different senders.
1:05	/arena or RESULTS.md	An agent that always picks the best quote: unprotected loses X, with FIRM loses nothing.
1:15	RESULTS.md	Measured on Monad Testnet: catch rate, zero false positives, gas overhead. Limits are in the threat model.

18. Risks and mitigations

Risk	Impact	Mitigation
Quoter cannot be used in-transaction, or its gas is too high (S1)	Core insight weakened or too costly	Fallback F1 with off-chain quote plus freshness; state the weaker guarantee plainly. Measure gas early; tune caps
Hook can recognise FirmRouter and behave honestly only toward it	Protection limited to hooks that do not discriminate	CanaryRunner probes from several senders; scoreboard shows probe-based evidence; document as an arms race
[NEW] Hook quotes high off-chain but honestly in a real transaction	User gets the low price without a strike	advertisedOut skip (8.4, P1); quote-api cross-checks
False positives on honest but unusual hooks	Unfair WATCH or BLOCK	Tight tolerance only after a strike; reinstate path; report shows evidence; WATCH rather than BLOCK for small shortfalls
Testnet reset or instability	Demo breaks	Redeploy.sh, recorded video, results saved in repo
RPC rate limits	Slow UI and agents	Caching, batching, multi-RPC rotation
Gas model (limit charged)	Fallback chains cost more	Gas caps per leg, MAX_LEGS = 4, report overhead honestly
[NEW] Seven days left	Unfinished core	7-day plan (16.1); cut P1/P2; fallback F9
[NEW] Missing disclosure or second submission	Disqualification	AI-tool statement in README; one project, one track
Others build similar ideas	Differentiation loss	Lead with measured evidence, the scoreboard standard and the canary; submissions are reviewed non-confidentially
Evidence disputes (0x vs Uniswap founder)	Credibility	Attribute precisely, avoid naming hooks, rely mainly on our own measured results
Centralised owner and attester roles	Trust criticism	State it in THREAT_MODEL.md; propose staked attesters as roadmap
Scope creep	Unfinished core	Follow P0/P1/P2; do not start BondVault before M7

19. References

Status: [V] verified (this pass or v1.0 research), [S] second-hand, [A] used for design assumptions to be confirmed by reading the source.

Reference	URL	Supports	Status
Metropolis landing page	https://monad.xyz/metropolis (and /developers/hackathons/metropolis)	Dates, four tracks, prizes, FAQ	[V]
Monad hackathons page	https://monad.xyz/developers/hackathons	Event index	[V]
Submission platform	https://hackathon.monad.xyz	Registration, submission, rules modal	[V]
Participant repos quoting rules v3.0	github.com/precious-akpan/monad-metropolis-merchant-rails; github.com/mandaputtra/ping-pong-pay (issue 10)	Testnet OK, open source mandatory, AI disclosure, one track	[S]
Testnet network information	https://docs.monad.xyz/developer-essentials/testnets	Chain ID, RPCs, canonical contracts, reset note	[V]
Monad changelog	https://docs.monad.xyz/developer-essentials/changelog	P256VERIFY (EIP-7951), revisions	[V]
Uniswap v4 hooks guide	https://docs.monad.xyz/guides/uniswap-v4-hooks (setup, deploy, pool-lifecycle, customize-and-publish)	Testnet walkthrough, addresses, tooling pins	[V]
Hooks example repo	https://github.com/monad-developers/uniswap-v4-hooks-example	Scripts, pinned deps, shared-infrastructure.json	[V]
Uniswap v4 deployments	https://docs.uniswap.org/contracts/v4/deployments	Mainnet Monad PoolManager	[V]
v4-core, v4-periphery	https://github.com/Uniswap/v4-core ; https://github.com/Uniswap/v4-periphery	Flag bits, Initialize event, settle/take, Quoter interface	[A]
V4Quoter reference	https://docs.uniswap.org/contracts/v4/reference/periphery/lens/V4Quoter	Revert-based, not for onchain calls	[V]
Uniswap hook registry	https://github.com/Uniswap/hooklist (hooks/monad)	Monad hooks list (P2 scan)	[V]
0x: Uniswap v4 hooks were a mistake	https://0x.org/post/uniswap-v4-hooks-were-a-mistake	84,163 hooks; 19.4 / 54.2 / 26.4 split; hook example	[V]
0x: PropAMM Shenanigans	https://0x.org/post/propamm-shenanigans	Quote spoofing patterns	[S]
Crypto Briefing coverage	https://cryptobriefing.com/0x-criticizes-uniswap-v4-hooks-malicious/	Uniswap founder's counter-arguments	[S]
The Monad Pulse #047	https://themonadpulse.substack.com/p/the-monad-pulse-047	Monad founder commentary	[S]
arXiv 2609.38056	https://arxiv.org/html/2609.38056	Monad propAMM share 12.5% to 45.7%; onchain router suggestion; gas-limit note	[V]
Envio: Monad Testnet	https://envio.dev/chains/monad-testnet ; https://docs.envio.dev/docs/HyperIndex/monad-testnet	Endpoints, config, free tier	[V]
Foundry book	https://book.getfoundry.sh	Toolchain	[V]
Model Context Protocol docs	https://modelcontextprotocol.io	MCP server design	[A]
MetaMask Agent Wallet plugins	https://docs.metamask.io/agent-wallet/plugins/	Plugin model; Early Access	[V]
Mera passkey library	https://monad.xyz/blog/introducing-mera	Client-side passkey accounts	[V]

Appendix A: CLAUDE.md (copy to the repository root)

```
# FIRM - project instructions for Claude Code

## What this is
FIRM is a Uniswap v4 router for Monad that checks each pool's quote against its real fill
inside the same transaction, rejects and publicly strikes pools that break the quote, and
falls back to the next pool. A Scoreboard contract records hook honesty. A canary agent
probes new hooks with dust swaps. Hackathon: Monad Metropolis, Track 1 (one track only).
Network: Monad Testnet (chain 10143). The full spec is in docs/FIRM-spec.pdf (v1.1). Follow it.

## Rules
- Work in milestone order (spec section 16) and follow the 7-day plan (16.1). Do the spikes
  in M0 first. Log in docs/SPIKES.md. Cut P1/P2 before P0. Never cut M7 or M8.
- Never invent addresses, ABIs, event signatures. Use deployments/10143.json and the spec's
  verified addresses. Confirm [A] items by reading v4-core / v4-periphery.
- Testnet only. No secrets in git. Use .env (see .env.example) or the Foundry keystore.
- Deterministic contracts decide money and status. LLMs only write prose or drive demo agents.
- Misbehaving test hooks live ONLY in contracts/test-fixtures. Never deploy to mainnet,
  never register in any hook registry.
- Every contract and service ships with tests. Run before commit. Commit small and often.
- Keep README.md, DEPLOYMENTS.md, RESULTS.md, THREAT_MODEL.md, docs/SPIKES.md current.
- README MUST state that AI coding tools (Claude Code) were used (hackathon rule).
- Be honest in docs about limits; do not round benchmark numbers in our favour.
- Do not replace the V4 Quoter with a self-simulation through FirmRouter (spec 4.2).

## Commands
- contracts: cd contracts && forge build && forge test -vvv
- deploy: cd contracts && ./script/Redeploy.sh
- services: pnpm -r build && pnpm -r test
- web: pnpm --filter web dev
- benchmark: pnpm bench

## Key facts
- Foundry v1.8.0, Solidity 0.8.26, EVM cancun. Gas on Monad is charged on the gas limit.
- Testnet Uniswap v4 is Monad-maintained, not official.
  PoolManager 0x451D64ab3b650040d2aE1886602b97ed6eDc643d
  V4Quoter    0x869834d127b230283fe63E0d0A9bEB67216a94C7
- Envio HyperSync for testnet: https://monad-testnet.hypersync.xyz
- Deadline: 13 Oct 2026, 11:59 PM ET. Submit by 12 Oct.
```

Appendix B: .env.example

```
# Foundry: prefer `cast wallet import firm-deployer --interactive` over a raw key.
DEPLOYER_ACCOUNT=firm-deployer
MONAD_TESTNET_RPC_URLS=https://testnet-rpc.monad.xyz
# Add fallbacks comma-separated: https://rpc.ankr.com/monad_testnet
# and https://rpc-testnet.monadinfra.com (mind each endpoint's rate limits)
ALCHEMY_MONAD_TESTNET_URL=
ENVIO_GRAPHQL_URL=
QUOTE_API_URL=http://localhost:8787
NEXT_PUBLIC_QUOTE_API_URL=http://localhost:8787
NEXT_PUBLIC_ENVIO_GRAPHQL_URL=
CANARY_PRIVATE_KEY=      # testnet only
TRADER_PRIVATE_KEY=      # testnet only
LLM_BASE_URL=            # any OpenAI-compatible endpoint
LLM_API_KEY=
LLM_MODEL=
SQLITE_PATH=./canary.sqlite
```

Appendix C: Glossary

Term	Meaning
Hook	A Uniswap v4 contract attached to a pool that runs custom logic during swaps and can change amounts

Term	Meaning
Quote	The expected output amount, here obtained from the V4 Quoter inside the swap transaction
Fill / settled output	The amount actually delivered when the swap executes
Shortfall (bps)	How far settled output falls below the quote, in basis points (1 bps = 0.01%)
Strike	A public record that a hook broke its quote beyond tolerance
Floor	Quote minus the tolerance for the hook's status; the minimum FIRM accepts for a leg
Scoreboard	Onchain ledger of hook statistics and status
Canary / probe	A small real swap used to test a hook before users trade
advertisedOut	**[NEW]** The off-chain quote passed with a route, compared with the in-transaction quote
PropAMM	Proprietary AMM: a market-maker-run pool that updates quotes actively
Spike	A short experiment that tests a risky assumption
Testnet reset	Wiping the testnet chain, which invalidates deployed contracts

Appendix D: Questions for the human, with defaults

Question	Default if no answer
Project name and branding	FIRM (tagline: Your quote is a promise)
Funded testnet keys for deployer, canary, trader	Required: Claude Code cannot proceed to M2 without them
LLM provider and key	Any OpenAI-compatible endpoint; if none, use template reports (fallback F6)
Envio hosted account and Vercel account	Needed for M3 and M4 hosting. If Envio is slow, use fallback F4
Licence	MIT
Include BondVault and passkeys?	No, until M7 is complete
[NEW] Confirm the 7-day plan and scope cut in 16.1	Yes. Follow it
[NEW] Confirm Track 1 as the single track	Yes (Track 4 is the alternative, but changing track is not planned)
[NEW] Which bounties' rules have been read on hackathon.monad.xyz?	Count none until read; Envio and an LLM provider are the likely ones